

A Controlled Architectural Ablation on SALICON for Saliency Prediction

Alessandro Martini 2178627[†], Edin Milenko 2178628[†]

Abstract—Modern saliency models have converged to performances that are very close to each other on standard benchmarks, and in this regime it is harder to identify which part of an architecture is responsible for its performance. This work takes an encoder-decoder architecture with several additional components and decomposes it, analyzing the effects of each addition under a protocol held fixed throughout, so that every difference between two configurations is attributable to a single cause. Starting from a plain ResNet18 encoder with a convolutional decoder, we successively add U-Net skip connections, gated self-attention on the two deepest encoder stages, and a deeper backbone. SIM, CC and KL improve monotonically at every step and NSS at every step but one, and the components do not contribute equally: the backbone swap alone contributes roughly four times as much correlation as the attention mechanism, while changing nothing else in the network. The attention module validates this from the inside, since its residual branches are governed by learnable gates and every gate closes relative to its initialization, further so for the deeper backbone, indicating that the network itself elects to use less attention as its encoder improves. The final configuration reaches CC 0.893, SIM 0.784 and KL 0.213 on a held-out test split, within the range of other SOTA architectures. Our results suggest that encoder capacity is the binding constraint on this task, and that effort invested downstream of it brings minor but useful performance increases.

Index Terms—Visual saliency, convolutional neural networks, self-attention, transfer learning, ablation study, encoder-decoder architectures.

I. INTRODUCTION

Human vision is selective: of the information falling on the retina at any instant, only a small fraction is processed at full resolution, and the visual system continuously decides where to direct that resolution next. Predicting those decisions, that is, computing a saliency map, has practical value wherever a system must decide where to spend limited resources, from perceptually weighted image compression to automatic cropping and the evaluation of user-interface layouts. The field was transformed by two developments. The first was data: SALICON [1] replaced the slow and expensive eye-tracking with a mouse annotation protocol, producing 20 000 annotated images where earlier datasets held a few hundred. The second was transfer, since features learned for object recognition apply to saliency almost directly [2]. Performance improved rapidly and then flattened: between the weakest and the strongest of the recent architectures we compare against [3], [4] lies roughly 0.03 in correlation coefficient. In such a

saturated regime, small decimal numbers stop being informative about *why* a model works. This paper asks that question instead of asking for another tenth of a point. Given a saliency architecture built from a pretrained encoder, a decoder, skip connections and an attention mechanism, how much does each part actually contribute? The question matters because the components are not equally expensive to obtain, a larger backbone being a one-line change while an attention module is a design decision with a cost in parameters and computation. Two findings follow, and they point the same way. Every component helps, but the backbone swap contributes roughly four times as much as the attention mechanism while changing nothing else in the network. Independently of that, the gates governing the attention branches, which we included so that training could switch the module off if it were harmful, close in every configuration and close further for the deeper backbone. Given the choice, the network elects to use less attention as its encoder improves. The contributions of this paper are as follows.

- A controlled architectural decomposition. Four configurations of a single architecture, trained under an identical protocol, so that the difference between any two rows is attributable to one component. SIM, CC and KL improve monotonically at every step and NSS at every step but one, and the per-step contributions are reported separately.
- Evidence that encoder capacity dominates. Replacing ResNet18 with ResNet50, with everything else held fixed, yields roughly four times the correlation gain of adding self-attention. The learned attention gates validate this from the inside by closing, and closing further for the deeper backbone.
- A reproducible pipeline, with negative results. We describe two preprocessing steps that fail silently if implemented naively, and report an unsuccessful adversarial-training experiment with a diagnosis of why it failed.

This report is structured as follows. Sec. II reviews the state of the art, Sec. III gives a high-level view of the pipeline and Sec. IV describes the data, its preprocessing and the metrics. The architecture and training strategy are detailed in Sec. V and the results in Sec. VI. Concluding remarks are provided in Sec. VII.

II. RELATED WORK

Early saliency models were built on explicit hypotheses about what attracts attention, the canonical example being Itti and Koch [5], which combines center-surround contrast

[†]Department of Information Engineering, University of Padova,
email: alessandro.martini.4@studenti.unipd.it
edin.milenko@studenti.unipd.it

computed on hand-designed channels into a single map. Such models need no training data but are limited by their own premise: they detect low-level contrast, whereas human gaze is drawn to semantics: a face attracts attention whether or not it is locally high-contrast.

Transfer from object recognition. What unlocked the deep approach was a dataset large enough to train on, SALICON [1], described in Sec. IV-A. DVA [2] showed that a network built on a VGG backbone and supervised on saliency substantially outperforms the classical models, establishing the finding our work builds on: features learned for object recognition transfer to saliency almost directly.

Changing the objective. A second line of work attacks the loss rather than the architecture. SalGAN [6] trains a saliency network adversarially, using a discriminator to judge whether a map is real or predicted, and reports a modest gain, CC 0.772 with binary cross-entropy against 0.786 with the adversarial term added. We implemented this approach and were unable to reproduce a benefit, for a reason consistent with SalGAN’s own numbers (Sec. VII-B).

Recent architectures. Current work pushes on capacity and on the form of the output. EML-Net [3] uses larger backbones (DenseNet, NasNet) and ensembles several of them, reaching CC 0.890 on SALICON validation, while MDNSal [4] predicts the parameters of a mixture of Gaussians rather than a dense map and reaches CC 0.899.

What is missing. These papers report end-to-end performance for complete systems, and the roughly 0.03 CC that separates them is small enough that attributing it to a specific design choice requires a controlled comparison rather than a leaderboard. What they do not report is a decomposition: given a fixed decoder and a fixed training protocol, how much of the performance comes from the encoder and how much from what is built on top of it. Ablations, when present, vary components within a single architecture rather than isolating the encoder itself.

III. PROCESSING PIPELINE

The task is dense regression, an RGB image in and a saliency map of the same size out, so the pipeline is an encoder-decoder. Two decisions shape everything that follows. First, the output is a distribution, not an image: a probability density over the image plane rather than a picture to be reproduced pixel by pixel, which determines the loss, the output representation and three of the four metrics. Second, *the features come from elsewhere*: rather than learning a visual representation from scratch on a small dataset, we take a pretrained ImageNet backbone and learn only the mapping from its features to a saliency map. The encoder is therefore not something we design but something we choose and build on top of. The resulting architecture (Fig. 2) chains preprocessing, encoding into a pyramid of feature maps at four resolutions, contextual refinement of the two deepest maps by gated self-attention, and decoding back to full resolution with skip connections. Our experiments *remove* components from this design one at a time, never replacing them, which

is what makes the comparison in Sec. VI-A interpretable: each configuration differs from the previous one in exactly one respect.

IV. SIGNALS AND FEATURES

A. The SALICON dataset

SALICON [1] is built on the images of MS-COCO [7] and collects attention data through a mouse-contingent paradigm: the image is shown blurred and the observer sharpens a small region around the cursor, so the cursor trace approximates the region of maximal attention. The annotation is noisier than eye-tracking but scales to tens of thousands of images, which is what makes training a deep network here possible. Each image comes with two distinct forms of ground truth, and the distinction matters because our metrics do not use the same one. The *density map* is continuous and real-valued in $[0, 1]$, obtained by convolving the recorded fixations with a Gaussian, and SIM, CC and KL are computed against it. The *fixation map* is the raw, discrete gaze coordinates of around sixty observers, stored as a sparse set of points; NSS is computed against it, since NSS asks how high the predicted saliency is exactly where humans looked. Confusing the two is an error that produces plausible numbers rather than a crash.

B. Dataset split

SALICON provides 10 000 training, 5 000 validation and 5 000 test images, but the labels for the official test set are held on the benchmark server. Published work therefore commonly reports figures on the validation set, which means the same data is used both to select the model and to report its performance. We avoid that. We keep the official training images for training and partition the 5 000 validation images into 3 500 for validation and 1 500 for test, using a fixed seeded shuffle identical across every experiment reported here. Model selection, early stopping and checkpoint choice use the validation set only; the test set is evaluated exactly once, at the end of each run. Every number in Tab. 2 is therefore measured on data that played no part in choosing the model.

C. Preprocessing

Images are resized to 256×256 and normalized with the standard ImageNet channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). The normalization is not cosmetic: the encoder was pretrained on inputs with exactly these statistics, and feeding it differently distributed data degrades the very features we rely on. Density maps are resized to the same size and kept in $[0, 1]$, with no normalization, being the regression target rather than a network input. Fixation maps require more care, because the obvious implementation is wrong. Rasterizing the fixation points at the original 640×480 resolution and then resizing to 256×256 destroys the data: a fixation map is a set of isolated points one pixel wide on a map of 307 200 pixels, so downsampling by a factor of 2.5 horizontally and 1.9 vertically makes most of them fall between output samples and disappear. NSS, which averages the predicted saliency over

the surviving points, is then computed on a corrupted ground truth. We instead build the binary map directly at the target resolution, rescaling the gaze coordinates before rasterizing:

$$x' = \left\lfloor x \cdot \frac{256}{W} \right\rfloor, \quad y' = \left\lfloor y \cdot \frac{256}{H} \right\rfloor, \quad (1)$$

where (W, H) is the original image size. Every recorded fixation survives, and the map is binary, a pixel being set if at least one observer fixated it, which is the standard input for NSS. Both approaches lose something, but not the same kind of thing: ours merges two fixations landing on the same output pixel, preserving the location and discarding only a duplicate, whereas resizing deletes isolated fixations outright depending on where they fall relative to the sampling grid, so a region attended by one observer can vanish entirely.

D. Data augmentation

Augmentation is applied to the training split only, jointly to all three maps, since a geometric transformation of the input must be matched on the target. We use horizontal flips ($p = 0.5$), which is safe because the statistics of human gaze are approximately left-right symmetric; rotations in $[-10^\circ, +10^\circ]$ ($p = 0.3$); and jitter on brightness, contrast and saturation drawn from $[0.8, 1.2]$ ($p = 0.5$), applied to the image only, since it changes the appearance of the scene but not where a human would look. Vertical flips and large rotations are excluded, human gaze not being symmetric under them. The rotation uses bilinear interpolation for the image and the density map but nearest-neighbor for the fixation map, which is the same trap as above in a different guise: bilinear interpolation spreads each point across four neighbors with fractional weights, and the map is no longer binary.

E. Metrics

We evaluate with the four metrics standard in the saliency literature, which are not redundant: each is sensitive to a different kind of error, so a model can improve on one while degrading on another. **SIM** is the histogram intersection $\sum_i \min(p_i, y_i)$ of the two maps normalized to sum to one, so it penalizes a prediction that misses a salient region far more than one that is merely diffuse. **CC** is the Pearson correlation, invariant to affine rescaling and therefore sensitive only to the spatial pattern. **NSS** standardizes the prediction and averages it over the fixation points only, rewarding peaks that fall precisely on real fixations; it is the most sensitive of the four to spatial sharpness and the only one evaluated against the fixation map. **KL**, our training objective, is strongly asymmetric, penalizing heavily any region where the ground truth places mass and the prediction does not.

V. LEARNING FRAMEWORK

The architecture is an encoder-decoder with a pretrained convolutional backbone, U-Net skip connections and gated self-attention on the two deepest encoder stages (Fig. 2).

A. Encoder

The encoder is a ResNet [8] pretrained on ImageNet, truncated before the global average pooling and the classification head. We keep the outputs of the four residual stages $c_1, \dots, c_4$, whose shapes are given in Fig. 2, since the decoder uses all of them. The property that matters is that the two backbones differ only in depth and width: the stages, strides and spatial resolutions are identical, ResNet50 using bottleneck blocks and therefore four times the channels of ResNet18 at every stage. Everything downstream keeps its structure: the decoder has the same number of stages and the same output widths, the attention operates on the same token counts with the same gating, and the training protocol is unchanged. What the swap alters is the width of the encoder features, and mechanically the input width of the layers that consume them, which is what makes the comparison in Sec. VI-A interpretable as a change of encoder representation rather than of architecture.

B. Gated self-attention

A convolutional encoder aggregates context only through its receptive field: two salient regions on opposite sides of the image influence one another only after enough layers have brought them within reach of the same kernel. Self-attention removes this constraint by letting every position attend to every other in a single operation, a natural fit for saliency, where the salience of a region depends on the rest of the scene. We apply multi-head self-attention [9] (8 heads) to c_3 and c_4 , flattening each feature map into a token sequence (256 and 64 tokens respectively), attending, and reshaping back. The two deepest stages are chosen because attention is quadratic in the number of tokens, so applying it at c_2 (1024 tokens) or earlier is prohibitively expensive at this input resolution. The block is residual and gated by a learnable scalar γ , as in Eq. (3), initialized at $\gamma = 0.1$. The gate serves two purposes. It guarantees a performance floor, since $\gamma = 0$ recovers the identity map exactly, so training can switch the module off if it is harmful, and it makes the module measurable, as exploited in Sec. VI-B.

C. Decoder

The decoder recovers the input resolution through five upsampling stages, the first four of which consist of a bilinear upsampling by a factor of two followed by a 3×3 convolution, batch normalization, dropout ($p = 0.2$) and a ReLU. Bilinear interpolation is preferred to transposed convolution, which produces artifacts highly visible on maps that are smooth by construction. In the full model the first three stages concatenate the corresponding encoder feature maps before the convolution, with an architecture inspired by U-Net [10]. The motivation is resolution: the encoder discards spatial detail at every downsampling stage, and by c_4 the representation is 8×8 , from which a decoder can recover *where* the salient regions roughly are but not their precise extent. The skips reinject that detail and shorten the gradient path to the encoder. The final convolution produces a single channel of raw logits,

TABLE 1: Training hyperparameters, identical for every configuration. Optimizer Adam, weight decay 10^{-5} , batch size 16, input 256×256 .

	Phase 1	Phase 2
Encoder / decoder / attention LR	– / 10^{-3} / –	10^{-5} / 10^{-4} / 10^{-3}
Max epochs, patience	6, 3	50, 5
LR schedule	none	plateau ($\times 0.5$, pat. 3)

with no sigmoid and no softmax inside the model, for the reason given next.

D. Loss

Losses that treat a saliency map as an image to be regressed pixel by pixel, such as mean squared error or per-pixel binary cross-entropy, are indifferent to whether the predicted map is a coherent distribution. We therefore train with the Kullback-Leibler divergence. Given the raw logits $\mathbf{z}$ of an image with N pixels and the ground-truth density map $\mathbf{y}$ normalized to sum to one,

$$\mathcal{L}_{\text{KL}}(\mathbf{z}, \mathbf{y}) = \sum_{i=1}^N y_i (\log y_i - \log p_i), \quad \mathbf{p} = \text{softmax}(\mathbf{z}), \quad (2)$$

where the softmax is taken over all N pixels jointly rather than per-pixel. This couples the predictions across the image, so raising the predicted saliency of one region necessarily lowers it elsewhere, the correct inductive bias for a distribution. KL is also among the metrics most widely reported in the literature, so training on it aligns the objective with the evaluation.

E. Training protocol

Training proceeds in two phases. In the first, the pretrained encoder is frozen and only the decoder is optimized: the decoder is randomly initialized, so its early gradients are large and uninformative, and propagating them into the encoder would destroy the pretrained features before the decoder has learned anything useful. In the second phase the whole network is unfrozen, with a different learning rate per parameter group reflecting how far each needs to move: the encoder is only nudged, the decoder is trained ten times faster, and the attention gates are given a rate high enough to leave their initialization within the epoch budget. Hyperparameters are listed in Tab. 1 and are identical across the four configurations.

The checkpoint retained is the one with the best validation loss reached during phase 2. In practice every configuration peaked between epochs 8 and 9 and stopped between epochs 13 and 14, so the four models were selected at comparable points along their training trajectories, a further reason why the differences in Sec. VI-A can be attributed to architecture rather than to unequal training.

VI. RESULTS

We evaluate the four configurations on the held-out test split of 1500 images, under the protocol of Sec. V-E. It is worth

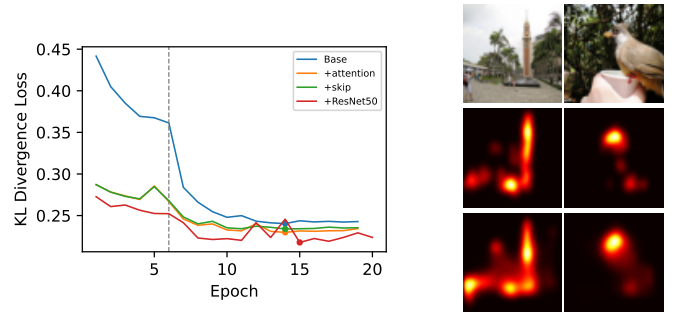


Fig. 1: Left: validation loss of the four configurations; the dashed line marks the end of phase 1. Right: input, ground truth and prediction of the final model.

TABLE 2: Test split results. Each row adds one component to the row above it; Δ is the change with respect to that row, not to the Base model. NSS is comparable across our configurations but not against published values (Sec. VI-C).

Model	SIM $\uparrow$	CC $\uparrow$	NSS $\uparrow$	KL $\downarrow$
Base	0.7679	0.8731	1.7936	0.2356
+ skip	0.7736	0.8799	1.8420	0.2309
+ attention	0.7758	0.8825	1.8419	0.2259
+ ResNet50	0.7844	0.8927	1.8634	0.2128
Δ skip	+0.0057	+0.0067	+0.0484	−0.0047
Δ attention	+0.0021	+0.0026	−0.0001	−0.0050
Δ ResNet50	+0.0086	+0.0103	+0.0215	−0.0132

first establishing a floor: human gaze is strongly center-biased, and a model that learned nothing but this bias would still obtain a superficially respectable score. On validation, a fixed center-prior Gaussian reaches CC 0.5354, NSS 0.9637 and KL 0.7932, against 0.8918, 1.8785 and 0.2177 for our model, so the network is predicting image-dependent structure rather than a static prior.

A. Architectural ablation

Tab. 2 reports the four configurations. The Base model uses a ResNet18 encoder and a decoder that consumes only the bottleneck; the second row adds the U-Net skip connections; the third adds self-attention on c_3 and c_4 ; the fourth changes nothing except the backbone. SIM, CC and KL improve monotonically at every step, and the improvement is not confined to the objective the network was trained on: KL is the training loss, but SIM and CC, neither of which is optimized directly, move in the same direction at every step. NSS improves at the skip and backbone steps but is flat at the attention step (−0.0001, below the sensitivity of our evaluation), which is consistent with the rest of the picture: the metric most sensitive to spatial sharpness does not respond to attention, and responds clearly to the backbone. Two observations follow. First, the skip connections give the largest single improvement in NSS (+0.0484), consistent with what they are for: NSS rewards precise localization of the peaks, and the skips restore

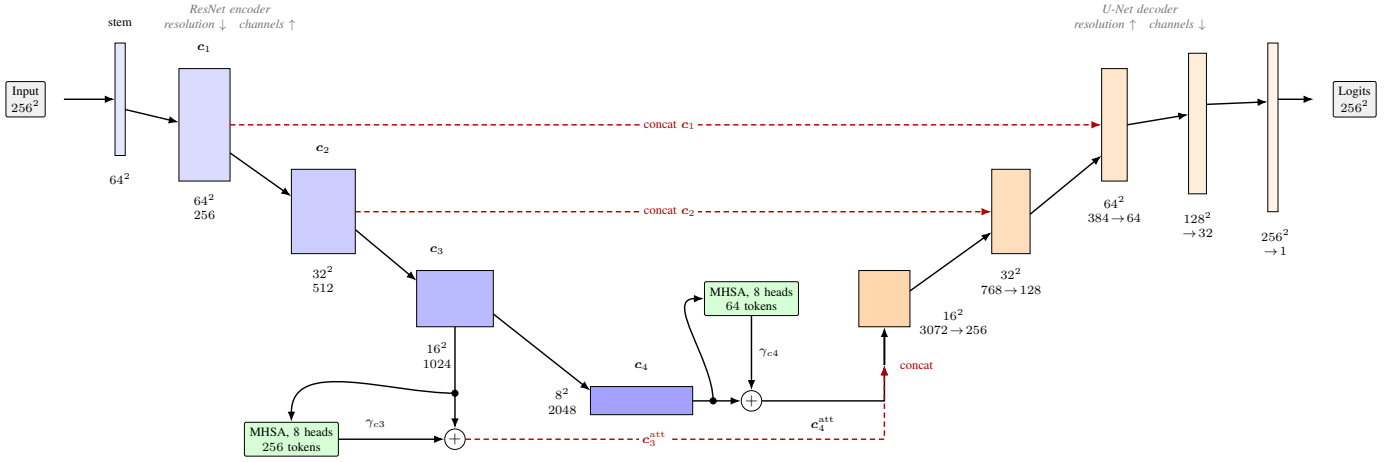


Fig. 2: SaliencyNet with the ResNet50 backbone. Block height encodes spatial resolution and block width channel count, both on a logarithmic scale, making the U shape visible: the encoder trades resolution for semantic depth and the decoder inverts the trade. Multi-head self-attention refines the two deepest stages as a residual branch scaled by a learned gate γ ; the first decoder stage concatenates the upsampled c_4^{att} with the skip c_3^{att} , hence its 3072 input channels.

TABLE 3: Learned attention gates. Both are initialized at 0.1 and trained with a learning rate of 10^{-3} , an order of magnitude above the decoder.

Model	γ_{c3}	γ_{c4}
Initialization	0.100	0.100
+ attention (R18)	0.084	0.018
+ ResNet50	0.070	0.002

exactly the detail the encoder discards. Second, and more importantly, the deeper backbone is the largest contributor to CC (+0.0103) and to KL (−0.0132), roughly four times the effect of adding attention on CC, and this is obtained without any change to the topology of the network, only to the width of its encoder features. The only thing that differs between the last two rows is the representational capacity of the encoder. This is the central result of our study: the binding constraint on this architecture is the encoder, not the machinery downstream of it.

B. The attention gates close

Each self-attention block is residual and gated by a learnable scalar,

$$c_{\text{out}} = c + \gamma \cdot \text{Attention}(c), \quad (3)$$

initialized at $\gamma = 0.1$. Since $\gamma \rightarrow 0$ recovers the skip-only model exactly, the learned γ measures how much the network actually wants to use the module, something the metrics alone cannot reveal. Despite a learning rate that gave the network wide freedom to increase them, every gate in Tab. 3 decreases from its initialization, and two patterns hold across both backbones. Within a model, the gate at the bottleneck closes far more than the one at the higher-resolution stage: on ResNet50 γ_{c4} is indistinguishable from zero, so the deepest attention block is effectively switched off. Across models,

both gates close further for the deeper backbone, γ_{c4} by an order of magnitude (0.018 to 0.002) and γ_{c3} from 0.084 to 0.070. The reading we propose is that self-attention here acts as a partial substitute for encoder capacity: its role is to let distant features interact, which a sufficiently deep encoder already achieves through its receptive field. The stronger the encoder, the less there is left for attention to add. This also explains why the attention step yields the smallest gain of the three: its contribution is small because the network keeps it small.

C. Limitations

Three caveats bound the claims above. Firstly, the backbone swap is not parameter-neutral. Widening the encoder features widens the layers that consume them: the first decoder convolution takes 3072 input channels instead of 768, and the attention blocks, whose cost grows with the square of the embedding dimension, grow substantially. The final configuration is therefore larger than the row it is compared against by more than the encoder alone accounts for. Secondly, each configuration was trained using a single seed, so we did not quantify seed variance and cannot attach a confidence interval to the individual runs. The gains from the deeper backbone and from the skip connections are large and consistent across all four metrics. The attention step is smaller (+0.0026 CC, −0.0001 NSS) and we regard it as consistent but not established, its NSS contribution in particular being of the same order as the numerical sensitivity of our own evaluation pipeline. Settling it would require a handful of runs at different seeds, which is the first thing we would do given more computation. Thirdly, NSS is not comparable with the literature. Both our loss and our metrics apply a softmax over the flattened map. NSS standardizes its input, $(x - \mu)/\sigma$, which makes it invariant to positive rescaling but not to the shape of the distribution it is given, and a softmax over 256×256 pixels has a very different

TABLE 4: SALICON validation. Published figures are taken from the respective papers. Our split is a fixed 3500-image partition of the official validation set (Sec. IV-B), so these figures are indicative rather than benchmark-certified.

Model	SIM $\uparrow$	CC $\uparrow$	KL $\downarrow$
EML-Net (DenseNet) [3]	0.769	0.873	0.230
EML-Net (NasNet) [3]	0.777	0.884	0.216
SaliencyNet (ours)	0.782	0.892	0.218
EML-Net (ensemble) [3]	0.785	0.890	0.204
MDNSal [4]	0.797	0.899	0.217

shape from the sigmoid-range map produced by BCE-trained models such as SalGAN [6]. Our NSS is therefore valid for comparing our own configurations, which is what Tab. 2 does, but not against published figures.

D. Comparison with published models

Tab. 4 places our final model against recent architectures on the SALICON validation set, on the three metrics computed identically across these works. Our model sits within the range of these architectures on all three metrics despite a considerably simpler design, a standard ResNet50 encoder with a U-Net decoder against the NasNet backbones of EML-Net and the mixture-density head of MDNSal. Since we evaluate on an internal split rather than the official benchmark server, these figures situate our result in the right range rather than placing it on a leaderboard.

VII. CONCLUDING REMARKS

We built a saliency predictor for SALICON as a controlled architectural ablation: four models under an identical protocol, each adding one component to the previous one. The final configuration reaches CC 0.893, SIM 0.784 and KL 0.213 on our held-out test split, within the range of considerably more elaborate published architectures. The result worth reporting is not the number but its decomposition. Of the three components, the one that matters most involves no architectural invention at all: replacing ResNet18 with ResNet50 contributes roughly four times as much CC as adding self-attention. The learned gates say the same thing from a different direction, closing further for the deeper backbone. On this task the binding constraint is the capacity of the encoder, and machinery placed downstream of it yields diminishing returns.

A. Future work

Two directions follow. If encoder capacity is the constraint, the obvious test is a ResNet101 or a transformer backbone, to see whether the trend continues or saturates. Second, our square 256×256 resize discards the original aspect ratio, so a higher-resolution or aspect-preserving input is another experiment that can be easily added and tested.

B. What we learned

Most of what we learned came from things that did not work. The experimental protocol is not overhead, it is the

experiment. For a long stretch of this project we compared models trained in separate runs, with different schedules and stopping points, and drew conclusions from differences in the third decimal place. Those conclusions were unstable: at one point we were convinced that self-attention was inert, on the basis of a comparison that turned out to be confounded. Only after re-running every configuration under one protocol did the picture become more clear. Adversarial training failed, and the failure was diagnosable. We implemented a SalGAN-style discriminator on top of our generator. It never learned to separate real from predicted maps, and the generator gained nothing. The reason, in hindsight, is that our generator was already too good: even with the weaker backbone, at CC ≈ 0.88 the predicted maps are close enough to the ground truth that the discriminator has no exploitable signal. Adversarial training helps when the base model produces maps that are obviously wrong, and ours did not. The technique was not wrong, our diagnosis was. The dangerous errors are the ones that do not throw. The two forms of SALICON ground truth are interchangeable as far as the type system is concerned, so feeding NSS the density map instead of the fixation map returns a plausible-looking number that is simply wrong. The same happens when downsampling a sparse fixation map. We lost time to both. Instrumenting the model is worth the effort. The gate γ was included only as a safety mechanism, but became the most informative single number we recorded, because it answers a question the metrics cannot: not whether the module helped, but whether the network wanted it.

REFERENCES

- [1] M. Jiang, S. Huang, J. Duan, and Q. Zhao, “SALICON: Saliency in context,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (Boston, MA, US), June 2015.
- [2] W. Wang and J. Shen, “Deep visual attention prediction,” *IEEE Transactions on Image Processing*, vol. 27, pp. 2368–2378, May 2018.
- [3] S. Jia and N. D. B. Bruce, “EML-NET: An expandable multi-layer network for saliency prediction,” *Image and Vision Computing*, vol. 95, p. 103887, Mar. 2020.
- [4] N. Reddy, S. Jain, P. Yarlagadda, and V. Gandhi, “Tidying deep saliency prediction architectures,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, (Las Vegas, NV, US), Oct. 2020.
- [5] L. Itti, C. Koch, and E. Niebur, “A model of saliency-based visual attention for rapid scene analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, pp. 1254–1259, Nov. 1998.
- [6] J. Pan, E. Sayrol, X. Giró-i-Nieto, C. Canton Ferrer, J. Torres, K. McGuinness, and N. E. O’Connor, “SalGAN: Visual saliency prediction with adversarial networks,” in *CVPR Scene Understanding Workshop (SUNw)*, (Honolulu, HI, US), July 2017.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, (Zurich, CH), Sept. 2014.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (Las Vegas, NV, US), June 2016.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, (Long Beach, CA, US), Dec. 2017.
- [10] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, (Munich, DE), Oct. 2015.